DEPARTMENT OF CSE

CS T61 ENTERPRISE SOLUTIONS

Unit-II – SAP AND ABAP/4

QUESTION BANK WITH ANSWER

SAP: History – SAP R/2 – SAP R/3 – Characteristics of SAP R/3 – Architecture of SAP R/3 - SAP Modules, NetWeaver, Customer Relationship Management, Business Warehouse, Advanced Planner and Optimizer. **ABAP/4**: Workbench - Workbench Tools - ABAP/4 Data Dictionary - ABAP/4 Repository Information – Structure of ABAP/4 program - ABAP/4 syntax – Data types – Constants and Variables – Statements : DATA, PARAMETERS, TABLE, MOVE, MOVE-CORRESPONDING, CLEAR, WRITE,CHECK, FORMAT. LOOP STRUCTURES. Sample programs.

PART A (2 MARKS)

1. Define SAP.

- Systems Applications and Products in Data Processing (SAP) is business software that integrates all applications running in an organization.
- These applications represent various modules on the basis of business areas, such as finance, production and planning, and sales and distribution, and are jointly executed to accomplish the overall business logic.
- SAP integrates these modules by creating a centralized database for all the applications running in an organization. We can customize SAP according to your requirements by using the Advanced Business Application Programming (ABAP) language.

2. Differentiate SAP R/2 and SAP R/3.

SAP R/2	SAP R/3
SAP R/2: is 2-tier architecture. In which all 3 layers [Presentation + Application +Database] are installed in two separate systems/server. (Server One – Presentation , Server Two – Application +Database)	SAP R/3: is 3-tier architecture. In which all 3 layers [Presentation + Application +Database] are installed in three separate systems/server. (server One -Presentation, server Two – Application, Server Three – Database)

3. Define SAP R/3.

- It is based on a client-server model, was officially launched on July 6, 1992. This version is compatible with multiple platforms and operating systems, such as UNIX and Microsoft Windows.
- SAP R/3 introduced a new era of business software—from mainframe computing architecture to a three-tier architecture consisting of the Database layer, the Application layer (business logic), and the Presentation layer.
- The SAP R/3 system contains various standard tables to execute various types of processes, such as reading data from the tables or processing the entries stored in a table.
- The SAP R/3 system uses these databases to handle the queries of the users

4. What are the types of views in SAP R/3 system?

The SAP R/3 system consists of the following three types of views:

- Logical view
- Software-oriented view
- User-oriented view

5. Name any five modules in SAP.

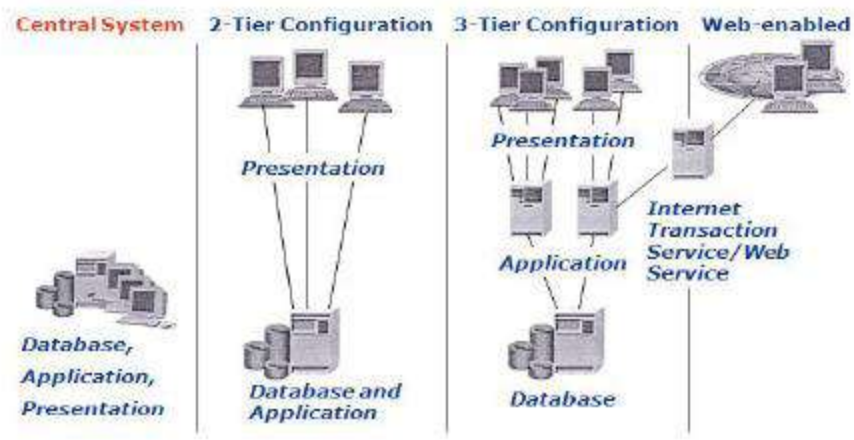
The SAP R/3 system provides the following set of applications, also known as functional modules, functional areas, or application areas:

- Financial Accounting (FI)
- Production Planning (PP)
- Material Management (MM)
- Sales and Distribution (SD)
- Controlling (CO)
- Asset Management (AM)
- Human Resources (HR)
- Project System (PS)
- Industry Solutions (IS)
- Plant Maintenance (PM)
- Quality Management (QM)
- Workflow (WF)

6. List the characteristics and issues of SAP R/2 with diagram.

- The SAP R/2 system was introduced in 1980.
- SAP R/2 was a packaged software application on a mainframe computer, which used the time-sharing feature to integrate the functions or business areas of an enterprise, such as accounting, manufacturing processes, supply chain logistics, and human resources.

- SAP R/2 system was based on a two-tier client-server architecture, where an SAP client connects to an SAP server to access the data stored in the SAP database.
- SAP R/2 was implemented on the mainframe databases, such as DB/2, IMS, and Adabas.
- The SAP R/2 system was designed to handle different languages and currencies.
- SAP R/2 system delivered a higher level of stability compared to the earlier version.



7. What is the purpose of NetWeaver?

- NetWeaver is an application builder from SAP for integrating business processes and databases from a number of sources while exploiting the leading Web services technologies.
- NetWeaver is getting a lot of industry attention as the first fully interoperable Web-based cross-application platform that can be used to develop not only SAP applications but others as well.
- NetWeaver allows a developer to integrate information and processes from geographically dispersed locations using diverse technologies, including Microsoft's .NET, IBM's WebSphere and Sun's Java technologies.

8. Write a note on VM Container.

- The VM Container technology enables a Java VM to be integrated into the ABAP work process.
- The execution of applications on the Java VM integrated in the ABAP work process is only intended for components developed by SAP.
- The VM Container technology in connection with a Java VM integrated in the ABAP work process

9. Define SAP CRM.

- SAP CRM is a part of SAP business suite. It can implement customized business processes, integrate to other SAP and non-SAP systems, and help achieve CRM strategies.
- SAP CRM can help an organization to stay connected to customers. This way organization can achieve customer expectations with the types of services and products that he or she actually needs.

10. What are the goals of CRM?

The goals of CRM are.

- Providing better and best customer service
- Selling products more effectively
- Discovering new customers and maintaining relation with existing customers.

11. List out the advantages of CRM.

- Increases customer satisfaction and attract new customers.
- Growth in number of customers in a high competition business world. It reduces cost and maximize profits.
- Boosts employee productivity and morale.

12. Draft a note on SAP business warehouse.

- SAP Business Warehouse (BW) is one of the important technical module of SAP and it stands for business information warehouse. The most important thing in the running of a business is data.
- Every business activity generates data. This data is used by a company is all areas of its business; It is used by the employees of the company in their work, to help them make a better decision.

13. What is meant by SAP APO?

- SAP APO stands for advanced planning and optimization and it is one of the most important component of SAP SCM (supply chain management).
- SAP is one the biggest vendor of business ERP application software in the world. One of the products that SAP has worked very hard on is the Supply Chain Management business suite, which helps a business to manage its supply chain better, while adjusting to constant changes in market conditions and holding on to their business advantage simultaneously.

14. Mention the important components of SAP APO.

The important components of SAP APO (advanced planning and optimization) are as follows.

- Demand Planning
- Supply Network Planning
- Production Planning and Detailed Scheduling (PP/DS)
- Global Availability and Available to Promise (ATP)

15. Define ABAP workbench.

- ABAP Workbench is a graphical programming environment in the SAP R/3 system to develop different applications by using the ABAP language.
- It provides different tools, such as ABAP Dictionary, ABAP Editor, and Screen Painter, to create ABAP applications.
- Using these tools, you can perform different tasks, such as defining data structures in ABAP Dictionary, developing data programs in ABAP Editor, and designing interfaces in Screen Painter and Menu Painter. Besides these tasks, you can also create user-defined reports, transactions, and enhancements within ABAP.

16. List out the different tools provided by ABAP workbench.

Different tools provided by ABAP Workbench:

- ABAP Dictionary
- ABAP Editor
- Class Builder
- Function Builder
- Screen Painter
- Menu Painter
- Object Navigator
- Maintain Message
- ABAP Text Elements
- Maintain Transaction

17. What is of ABAP dictionary?

- ABAP Dictionary is one of the important tools of ABAP Workbench. It is used to create and manage data definitions without redundancies.
- ABAP Dictionary always provides the updated information of an object to all the system components.

- The presence and role of ABAP Dictionary ensures that data stored in an SAP system is consistent and secure.

18. Draft a note on ABAP Front-End New and Old editor.

Front-End Editor (New)

- Front-End Editor (New) is the latest programming tool to create applications in an SAP system.
- It provides various new features, such as code hints, syntax highlighting, and auto-completion of language structures and elements

Front-End Editor (Old)

- As stated earlier, Front-End Editor (New) is used to edit the source code of the development objects.
- The classic mode of Front-End Editor (Old) is also used to edit the development objects, such as ABAP programs, function modules, and type groups. The source code of a program in Front- End Editor (Old) appears in plain text.

19. Write the ABAP syntax.

- ABAP programs are made up of individual statements.
- The first word in the statement is called an ABAP keyword.
- Each statement must end with a period.
- Statements can be intended.
- Can have multiple statements in a single line.

20. List the different types of ABAP data types.

The following are the different types of data type. They are

- Data elements
- Structure
- Table Types

21. Write a note on ABAP/4 repository information.

- The description of the ABAP Repository Information System given here is restricted to aspects of relevance for work with the Data Modeler. Using the ABAP Repository Information System, you can search for all modeling objects of the Data Modeler that you wish to display or edit.
- The ABAP Repository Information System provides you with the two basic functions Find and Where-used list.
- The Find function allows you to find objects of a specific object class.
- The Where-used list function allows you to determine the other objects in which a particular object is used.

22. Define MOVE statement and write the syntax for MOVE statement.

- The MOVE statement is used to assign the value of a data object to another data object; that is, to transfer the content of one field to another.

  Syntax:

  MOVE <f1> TO <f2>.

  Where <f1> is the data object whose data has to be transferred to another data object, <f2>. The equivalent statement for the MOVE statement is <f2> = <f1>.
- MOVE statement to assign values to multiple data objects, such as <f4> = <f3> =<f2> = <f1>.

  Syntax of MOVE statement for multiple assignments:

  Move <f1> TO <f2>

  Move <f2> TO <f3>

  Move <f3> TO <f4>

23. What is the purpose of CLEAR statement?

- It is used to reset the value of a data object. Resetting the values of the data objects depends on the data type of the data objects.

  CLEAR <f>.

  In this syntax, <f> is either a field or a field string.

24. Write a sample program for CLEAR statement.

The sample program:

```
REPORT ZCLEAR.
*---------------------------------------*
*/ Data Declarations
*/
DATA number TYPE I VALUE '80'.
*---------------------------------------* WRITE number.
*---------------------------------------**
/Using CLEAR statement
CLEAR number. WRITE / number.
*---------------------------------------*
```

25. Describe WHILE statement and show the sample program using WHILE statement.

- Conditional loops execute several statements only after certain conditions are fulfilled. This can be accomplished with the WHILE statement.
- The WHILE statement allows you to execute a block of statements so long as the condition mentioned in the WHILE statement evaluates to true.
- The syntax of the WHILE statement is:

```
WHILE <condition> [VARY <f> FROM <f1> NEXT <f2>]
        [statement_block]
ENDWHILE.
```

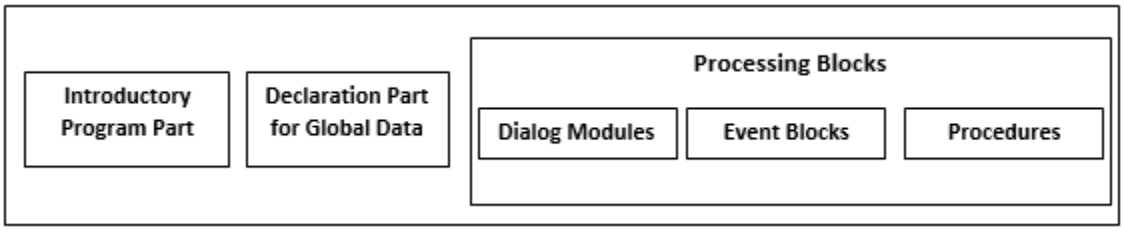
Sample program

```
REPORT ZWHILE_DEMO.
*-----------------------------------*
*/ Data Declarations
*/
DATA: LENGTH TYPE I VALUE 0, VAR1 TYPE I VALUE 0,
TEXT1 (50) TYPE C VALUE 'Calculating the Length of the Text
String'.
*-----------------------------------* VAR1 = STRLEN (TEXT1).
*-----------------------------------*
*/Using WHILE statement WHILE TEXT1 NE SPACE. WRITE TEXT1 (1).
LENGTH = SY-INDEX.
SHIFT TEXT1. ENDWHILE.
*-----------------------------------* WRITE: / 'STRLEN: ',
VAR1.
```

WRITE: / 'Length of string:' LENGTH.

26. Show the structure of ABAP program.

- The structure of an ABAP program includes the following:
- Introductory program part
- Global declaration part
- Processing blocks (consisting of different functions, such as procedures, dialog box, and event blocks) below figure shows the structure of an ABAP program:



27. Mention the different types of ABAP programs.

- The program types that can be selected are:
    1. Executable programs
    2. Module pools
    3. Subroutine pools
    4. Include programs
- Besides the preceding program types, some other types of ABAP programs exist that are not created by ABAP Editor. Instead, ABAP Workbench provides specific tools to create and maintain these programs. These program types are:
    1. Function pools
    2. Class pools
    3. Interface pools

28. What are the statements are used to declare variable statically in ABAP?

The statements that declared variable statically are

- DATA
- STATICS
- CLASS-DATA

- PARAMETERS
- SELECT- OPTIONS
- RANGES

29. Write the syntax for CONSTANT statement.

- Constants are named data objects created statically by using declarative statements

  The **syntax** of the CONSTANTS statement is:

  CONSTANTS <f> ... [TYPE <type>|LIKE <obj>]... [VALUE <val>].

- Notice that the syntax of the CONSTANTS statement is similar to the DATA statement.

30. List out the different types of FORMAT statements available in ABAP and write its syntax.

- Different types of syntaxes are available for specific actions performed with the FORMAT statement

FORMAT Statement	Description
FORMAT <option1> [ON|OFF] <option2> [ON|OFF].....	Specifies that the formatting options are set statically in a program. The option mentioned in the <option> expression is applicable to all output until the formatting option is turned off by using the OFF clause, as indicated in the statement. Note that the ON or OFF clause is optional.
FORMAT <option1> = <var1> <option2> = <var2>....	Specifies that the formatting options are set dynamically at runtime. The variables var1, var2, ... are interpreted as numbers and should be declared with the data type as I. If the content in the var1 and var2 variables is zero, the variable has the same effect as the OFF clause. If the numbers in the var1 and var2 variables are not equal to zero, either the color is shown according to the numbers stored in the variables or the variables have the same effect as that of the ON clause. Note that the formatting options are all set to their default values whenever they are used for the new event.
FORMAT RESET	Sets all the formatting options to off.

PART – B (11 MARKS)

1. Explain the history of SAP.

History of SAP Systems

- SAP is a translation of the German term Systeme, Anwendungen, und Produkte in der Datenverarbeitung.
- It was developed by SAP AG, Germany.
- The basic idea behind developing SAP was the need for standard application software that helps in real-time business processing.
- The development process began in 1972 with five IBM employees: Dietmar Hopp, Hans-Werner Hector, Hasso Plattner, Klaus Tschira, and Claus Wellenreuther in Mannheim, Germany.

SAP R/1 Systems

- A year later, the first financial and accounting software was developed; it formed the basis for continuous development of other software components, which later came to be known as the SAP R/1 system.
- Here, R stands for real-time data processing and 1 indicates single-tier architecture, which means that the three networking layers, Presentation, Application, and Database, on which the architecture of SAP depends, are implemented on a single system.
- SAP ensures efficient and synchronous communication among different business modules, such as sales and distribution, production planning, and material management, within an organization.
- These modules communicate with each other so that any change made in one module is communicated instantly to the other modules, thereby ensuring effective transfer of information

The SAP R/2 system

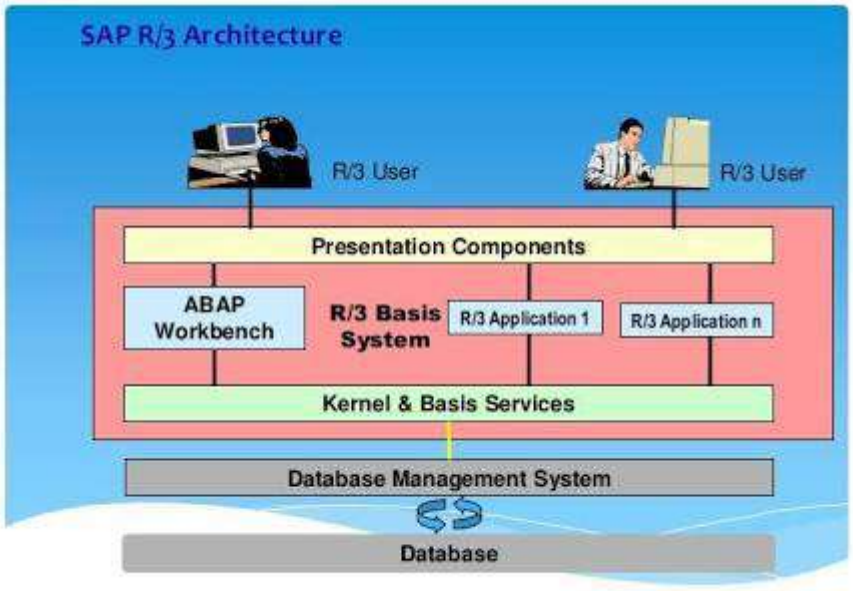
- It was introduced in 1980.
- SAP R/2 was a packaged software application on a mainframe computer, which used the time-sharing feature to integrate the functions or business areas of an enterprise, such as accounting, manufacturing processes, supply chain logistics, and human resources.
- The SAP R/2 system was based on a two-tier client-server architecture, where an SAP client connects to an SAP server to access the data stored in the SAP database.
- Keeping in mind that SAP customers belong to different nations and regions, the SAP R/2 system was designed to handle different languages and currencies.

SAP R/3 Systems

- It is based on a client-server model, was officially launched on July 6, 1992.
- This version is compatible with multiple platforms and operating systems, such as UNIX and Microsoft Windows.
- SAP R/3 introduced a new era of business software—from mainframe computing architecture to a three-tier architecture consisting of the Database layer, the Application layer (business logic), and the Presentation layer.
- The three-tier architecture of the client-server model is preferred to the mainframe computing architecture as the standard in business software because a user can make changes or scale a particular layer without making changes in the entire system.
- The SAP R/3 system is a customized software with predefined features that you can turn on or off according to your requirements.
- The SAP R/3 system contains various standard tables to execute various types of processes, such as reading data from the tables or processing the entries stored in a table.
- The SAP R/3 system uses these databases to handle the queries of the users.
- With the passage of time, a business suite that would run on a single database was required. This led to the introduction of the mySAP ERP application as a follow-up product to the SAP R/3 system.

2. Discuss in detail about architecture of SAP R/3.

- As stated earlier, the SAP R/3 system evolved from the SAP R/2 system, which was a mainframe.
- The SAP R/3 system is based on the three-tier architecture of the client-server model.
- SAP R/3 architecture shows how the R/3 Basis system forms a central platform within the R/3 system. The architecture of the SAP R/3 system distributes the workload to multiple R/3 systems.



SAP R/3 Architecture

- The link between these systems is established with the help of a network.
- The SAP R/3 system is implemented in such a way that the Presentation, Application, and Database layers are distributed among individual computers in the SAP R/3 architecture.
- The SAP R/3 system consists of the following three types of views:
    1. Logical view
    2. Software-oriented view
    3. User-oriented view

The Logical View

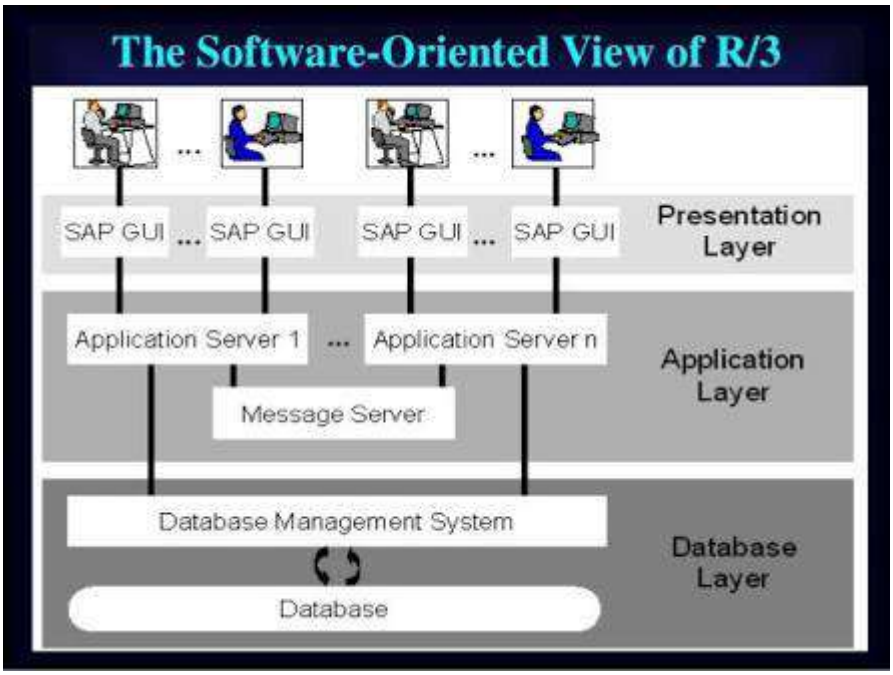
- The logical view represents the functionality of the SAP system. In this context, the R/3 Basis component controls the functionality and proper functioning of the SAP system.
- Therefore, in the logical view of the SAP R/3 system, we describe the services provided by the R/3 Basis component that help to execute SAP applications.

The following is a description of the various services provided by the R/3 Basis component:

- **Kernel and Basis services**—provide a runtime environment for all R/3 applications.
  The tasks of the Kernel and Basis services are as follows:
  - Executing all R/3 applications on software processors (virtual machines).
  - Handling multiple users and administrative tasks in the SAP R/3 system
  - Accessing the database in the SAP R/3 system.
  - Facilitating communication of SAP R/3 applications with other SAP R/3 systems and with non- SAP systems.
  - Monitoring and controlling the SAP R/3 system when the system is running.
- **ABAP Workbench service**—provides a programming environment to create ABAP programs by using various tools, such as the ABAP Dictionary, ABAP Editor, and Screen Painter.
- **Presentation Components service**—Helps users to interact with SAP R/3 applications by using the presentation components (interfaces) of these applications.

The Software-Oriented View

- The software-oriented view displays various types of software components that collectively constitute the SAP R/3 system.
- It consists of SAP Graphical User Interface (GUI) components and Application servers, as well as a Message server, which make up the SAP R/3 system.
- Since the SAP R/3 system is a multitier client-server system, the individual software components are arranged in tiers.
- These components act as either clients or servers, based on their position and role in a network.



- Software-oriented view of the SAP R/3 system consists of the following three layers:
    1. Presentation layer
    2. Application layer
    3. Database layer

Presentation Layer

- The Presentation layer consists of one or more servers that act as an interface between the SAP R/3 system and its users, who interact with the system with the help of well-defined SAP GUI components.
- For example, using these components, users can enter a request, to display the contents of a database table.
- The Presentation layer then passes the request to the Application server, which processes the request and returns a result, which is then displayed to the user in the Presentation layer.

Application Layer

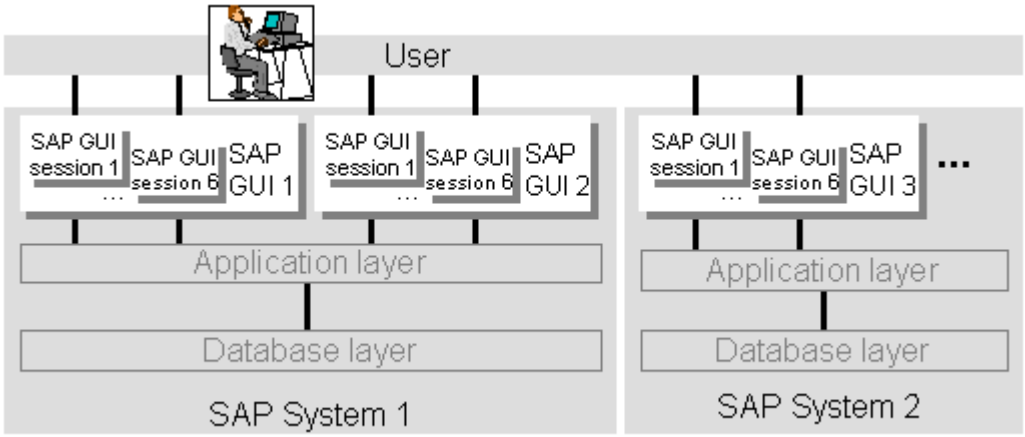
- The Application layer executes the application logic in the SAP R/3 architecture.
- This layer consists of one or more Application servers and Message servers.
- Application servers are used to send user requests from the Presentation server to the Database server and retrieve information from the Database server as a response to these requests.
- Application servers are connected to Database servers with the help of the local area network

Database Layer

- The Database layer of the SAP R/3 architecture comprises the central database system.
- The central database system has two components, DBMS and the database itself. The SAP R/3 system supports various databases, such as Adabas D, DB2/400 (on AS/400), DB2/Common Server, DB2/MVS, Informix, Microsoft SQL Server, Oracle, and Oracle Parallel Server.
- The database in the SAP R/3 system stores the entire information of the system, except the master and transaction data.

The User-Oriented View

- The user-oriented view displays the GUI of the R/3 system in the form of windows on the screen.
- These windows are created by the Presentation layer.
- To view these windows, the user has to start the SAP GUI utility, called the SAP Logon program, or simply SAP Logon.
- After starting the SAP Logon program, the user selects an SAP R/3 system from the SAP Logon screen.



User Oriented View

- The SAP Logon program then connects to the Message server of the R/3 Basis system in the selected SAP R/3 system and retrieves the address of a suitable Application server; i.e., the Application server with the lightest load.
- The SAP Logon program then starts the SAP GUI connected to the Application server.

- The SAP GUI starts the logon screen. After the user successfully logs on, the initial screen of the R/3 system appears. This initial screen starts the first session of the SAP R/3 system. Figure 1.5 shows the user-oriented view of the SAP R/3 system:
- A user can open a maximum of six sessions within a single SAP GUI. Each session acts as an independent SAP GUI. You can simultaneously run different applications on multiple open R/3 sessions. The processing in an opened R/3 session is independent of the other opened R/3 sessions.

3. Detail about Customer Relationship Management, Business Warehouse and Advanced Planner and Optimizer.

Customer Relationship Management:

What is SAP CRM?

- SAP CRM stands for customer relationship management. SAP CRM module is one of the best tool provided by SAP and supports customer related process end to end.
- The main concept of SAP CRM is managing and maintaining relationship with customers by firms. It deals with acquisition of customers and to its final termination.
- SAP CRM enable a firm to obtain a 360 degree over there customers and various functions such as accounts receivable (A/R), invoicing, fulfilment, delivery, decision making and various functions.
- SAP CRM module is integrated with other SAP modules such as SCM (Supply chain management), PLM (Product life cycle management), and SRM (Supplier relationship management) and with various modules.

Why CRM

- To have good relationship with customers and attract new customers.
- Effective Communication with customers.
- To increase the sales and maximize the profits.
- Analyzing customers, vendors and partners.

Goals of CRM:

- Providing better and best customer service
- Selling products more effectively
- Discovering new customers and maintaining relation with existing customers.

Versions:

- SAP CRM 7.0 released in 2009
- SAP CRM 6.0 released in 2007
- SAP CRM 5.0 released in 2006

ERP with SAP CRM

- SAP CRM is one of the import tool in SAP ERP functionality.
- CRM enables the firms to know about their customers and maximize profit and expand the revenue.
- It enable trade promotional management to improve visibility and control in to the trade promotion process

CRM key Features

- SAP CRM consists of three sub modules – Marketing, Sales & Services

Advantages of CRM

- Increases customer satisfaction and attract new customers.
- Growth in number of customers in a high competition business world. It reduces cost and maximize profits.
- Boosts employee productivity and morale.

Business Warehouse:

- SAP BW is one of the important technical module of SAP and it stands for business information warehouse. The most important thing in the running of a business is data. Every business activity generates data. This data is used by a company is all areas of its business; It is used by the employees of the company in their work, to help them make a better decision.
- SAP business information warehouse shortened to business warehouse are BW is a package comprehensive business intelligence product center on a data warehousing platform.

USES:

- Optionally designed for quick response time.
- Improves overall processing performance.
- No possibility of corrupting data.

SAP BW Provides the Following:

- Data warehousing system with optimized data structures for reporting and analysis

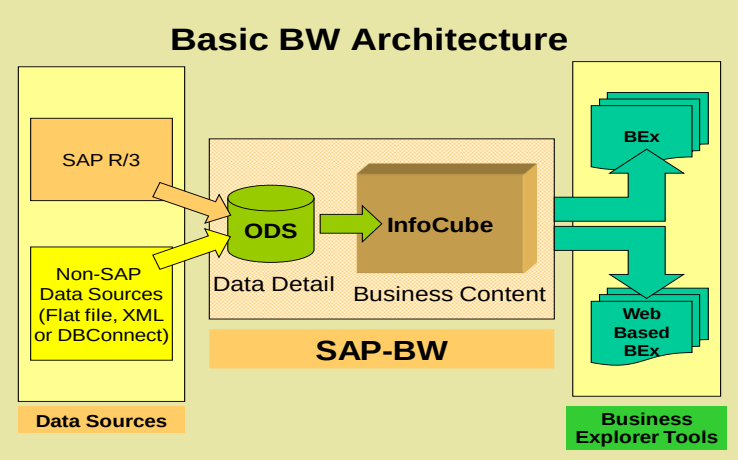
- Separate system
- OLAP engine and tools
- Comprehensive data-warehousing architectural base
- Automated data warehouse management
- Pre configuration using SAP global business know-how.

SAP BW Tools

- SAP BW has most comprehensive tools, business processes and functions for access and visualization.
- These tools enable the executive working with SAP BW to display the detailed information gathered by him along with extensive – planning data and business data analyses. He can do so in exhaustive detail or as a brief overview depending on how he wishes to present the data. Plus, he can present the data in any work environment of his choice – this could be in the form of Web based data or Microsoft Excel.
- The SAP BW tools enable the flexible distribution of the most vital data to all individuals involved in the process of decision making. All individuals can be notified of the latest information by email through a corporate eportal.

BW Reporting Tools

- MS-Excel-Based Analyzer (Web or GUI)
- Formatted Reporting
- Web Application Designer



Terminology and Object in SAP BW InfoObjects

- Business analysis-objects(customer, sales volume, and so on)are called infoObject in SAP BW
- Characteristics can be further divided into units, time characteristics, and technical characteristics.
- Key figures are all data fields that are used to store values or quantities.

Terminology and Objects in SAP BW InfoCube

- The central data containers that form the basis for reports and analyses in SAP BW are called infoCubes.
- They contain key figures (sales volumes, incoming order, actual characteristics (that is, to the master data of the SAP BW system [cost center, customers and so on).

Terminology and Objects in SAP BW InfoProvider

- InfoProvider is the super-ordinate term for an object that you can use to create reports in Business Explorer (BEx). InfoProviders deliver data from physical data stores..
    - InfoCubes
    - InfoObjects
    - ODS Objects

  or logical views of physical data stores…
    - InfoSets
    - Remote Cubes
    - Virtual InfoCubes
    - MultiProviders

Terminology and Objects in SAP BW Operational Data Store (ODS)

- ODS is a data store in which data is stored at a basic lever (document lever).
- This data is often from various data sources and/or source systems.
- ODS objects should be reported very selectively because ODS objects are not built for reporting purposes other than in drill-down paths to retrieve a few detailed records.

Advantages of SAP BW

- Every business seeks a competitive advantage, an "XFactor" to give it a leg-up over the competition. SAP BW gives a business that advantage. This is because SAP BW does critical business functions, such as reporting, analysis and interpretation of business data.
- It helps a business to optimize its business processes and access the most important and most relevant data pertaining to the most crucial aspects of the business within seconds.
- The several tools available with SAP BW allows a business to integrate data from several SAP modules, , transform it into a more legible and easily understandable form, and consolidate data from various sources into one solid, concrete information that throws light on the intricacies of a business process.
- The SAP BW provides a well thought out, high performance, well-structured infrastructure that allows the end user to evaluate and interpret complex data, simplify it and to decipher its meaning.

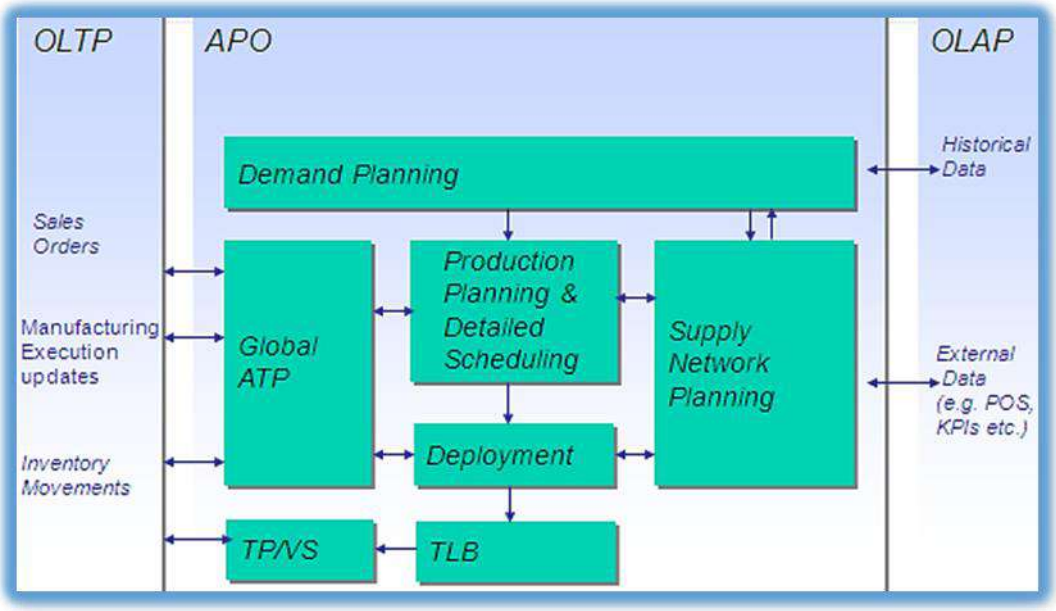
SAP Advanced Planner and Optimizer:

- Advanced Planning and Optimization (APO) is one of the most important module of SAP.
- SAP APO offers a fully integrated pallet of functions that you use to plan and execute your supply chain processes.
- SAP is one the biggest vendor of business ERP application software in the world.
- One of the products that SAP has worked very hard on is the Supply Chain Management business suite, which helps a business to manage its supply chain better, while adjusting to constant changes in market conditions and holding on to their business advantage simultaneously.

SAP APO supports the following:

- Business collaboration on a strategic, tactical, and operational planning level.
- Co-operation between partners at all stages of the supply chain process; from order receipt, stock monitoring, through final shipping of the product.

- Cultivation of customer and business partner relationships.
- Constant optimization and evaluation of the supply chain network's efficiency.



APO Architecture

Modules of APO

The sub modules of Advanced Planning and Optimization are:

1. Demand Planning.
2. Supply Network Planning / CIF.
3. Production Planning / Detailed Scheduling.
4. Global Available to promise / Deployment.
5. Transportation Management.

Demand Planning

- This is a subset of Demand Management.
- It is a business planning process that enables sales teams (and customers) to develop demand forecasts and inputs to feed various planning processes, production, inventory planning, procurement planning and revenue planning.
- Based on estimated product demand, a firm can plan for deployment of its resources to meet this demand.

- It is a bottom-up process as different from any top-down management process.
- It is also seen as a multistep operational SCM process to create reliable demand forecasts.
- Effective demand planning helps to improve accuracy of revenue forecasts, align inventory levels in line with demand changes and enhance product-wise/channel-wise profitability.
- Its purpose can be seen as to drive the supply chain to meet customer demands thro effective management of company resources.

Product Planning/Detailed Scheduling

The Production Planning and Detailed Scheduling (PP/DS) component:

- To create procurement proposals for in-house production or external procurement to cover product requirements.
- To optimize and plan the resource schedule and the order dates/times in detail.
- You can take the resource and component availability into account here.
- Reduce lead times
- Increase on-time delivery performance
- Increase the throughput of products and reduce the stock costs, through better coordination of resources, production, and procurement

Supply Network Planning (CIF)

- Supply planning, interactive supply planning, integration with other SCM components.
- Planning methods: Heuristics, optimization and CTM.
- Deployment.
- Transport load builder.
- Releasing supply plans to DP, PPDS.
- CIF core interface
- Monitoring and handling CIF errors.
- Comparison and reconciliation.
- Initial and change transfers, background jobs.

Global Available To Promise

- Global ATP is a type of availability check and business function that provides a response to customer order inquiries, based on resource availability.

- Execution of an ATP run generates available quantities of the requested product and delivery due dates.
- This helps companies support order promising and fulfillment, aiming to manage demand and match it to production and procurement plans while maximizing on customer service.

Transportation Management

- Purpose
    - Moving goods from one plant to another
    - Transportation planning
- Benefits
    - Transportation execution monitoring
- Key Process Steps
    - Create Stock Transfer Order
    - Creating an Outbound Delivery for Purchase Orders
    - Checking the Generated Outbound Delivery
    - Generating a Shipment Document
    - Creating a New Transfer Order Order with the Same Plant
    - Creating an Outbound Delivery for the New Stock Transfer Order
    - Adding a New Delivery to the Existing Shipment
    - Maintaining Delivery Picking and Posting Goods Issue
    - Goods Receipt with reference to Purchase Order

Benefits of Advanced Planner and Optimizer

- Reduction in costs due to simulation, 'what-if' capabilities.
- Integrating companies in the supply chain.
- Simultaneous planning of the problem to reach a synchronized sales, procurement, distribution and transport planning across the supply chain.
- Robust Supply Chain modeling capabilities.
- Decision making capabilities for the optimal location of facilities for the supply chain network
- All modules (DP/SNP/GATP/PPDS) are tightly integrated so that the supply chain entities can change their plans according to the needs of the market and this change can be seamlessly integrated across all the module.

- Tightly Integrated with ECC using Core Interface – enabling easy sync between planning and execution.

4. Describe about ABAP/4 workbench tools.

ABAP WORKBENCH

- ABAP Workbench is a graphical programming environment in the SAP R/3 system to develop different applications by using the ABAP language.
- It provides different tools, such as ABAP Dictionary, ABAP Editor, and Screen Painter, to create ABAP applications.
- Using these tools, you can perform different tasks, such as defining data structures in ABAP Dictionary, developing data programs in ABAP Editor, and designing interfaces in Screen Painter and Menu Painter. Besides these tasks, you can also create user-defined reports, transactions, and enhancements within ABAP

ABAP WORKBENCH TOOLS

- The ABAP Workbench tools are used to write ABAP code, design the screens, and create user interfaces. The following are different tools provided by ABAP Workbench.
    1. ABAP Dictionary
    2. ABAP Editor
    3. Class Builder
    4. Function Builder
    5. Screen Painter

6. Menu Painter
7. Object Navigator
8. Maintain Message
9. ABAP Text Elements
10. Maintain Transaction

1. ABAP Dictionary

- ABAP Dictionary is one of the important tools of ABAP Workbench.
- It is used to create and manage data definitions without redundancies. ABAP Dictionary always provides the updated information of an object to all the system components.
- The presence and role of ABAP Dictionary ensures that data stored in an SAP system is consistent and secure.
- The data definitions stored in ABAP Dictionary allow you to create various objects, such as tables and views.
- The objects stored in ABAP Dictionary are logically related to each other in the context of an application

The initial screen of ABAP Dictionary provides the following object types.

- **Database table**— Creates table definitions in ABAP Dictionary, independent of any database. The created table definition can then be used to create a table (with the same table structure) in the database.
- **View**— Creates view definitions in ABAP Dictionary. A view is a virtual table that does not store the data physically. It has a logical structure that represents or points to the data from one or more database tables. From the created view definition, you can create a view (with the same view structure) in the database.
- **Data type**— creates a definition of a user-defined type in ABAP Dictionary. The created definition of a user-defined type can then be used in a program to define data types and data objects.
- **Type group**— Creates groups of data types in ABAP Dictionary.
- **Domain**— Creates domains in ABAP Dictionary. A domain is used to describe the technical attributes of a field, such as a range of values and the type of data that are acceptable in a field.
- **Search help**— creates a help document (F4 help) or called input help for fields. A help document related to a field provides you help to enter a set of valid values in the field.

- **Lock object**— creates a local or lock object that helps synchronize the access of multiple users simultaneously. This is because an SAP system does not allow multiple users to access an object simultaneously. In such a situation, ABAP Dictionary allows you to create a lock or local object.

2. ABAP Editor

- ABAP Editor is used to create ABAP programs by writing code. You can use the ABAP Editor to define the class methods, function modules, screen flow logic, type groups, and logical databases for the ABAP Programs.
- ABAP Editor is available in three different modes for the front-end and back-end purposes.
    1. Front-End Editor (New)
    2. Front-End Editor (Old)
    3. Back-End Editor

Front-End Editor (New)

- Front-End Editor (New) is the latest programming tool to create applications in an SAP system.
- It provides various new features, such as code hints, syntax highlighting, and auto-completion of language structures and elements

Front-End Editor (Old)

- As stated earlier, Front-End Editor (New) is used to edit the source code of the development objects.
- The classic mode of Front-End Editor (Old) is also used to edit the development objects, such as ABAP programs, function modules, and type groups.
- The source code of a program in Front- End Editor (Old) appears in plain text.

Back-End Editor

- Back-End Editor, another classic editor, allows users to edit ABAP code.
- This editor is line-based and is used to perform normal editor functions, such as cut, copy, and paste.

3. CLASS BUILDER

Class Builder is a tool of ABAP Workbench used to create, define, change, and test the global ABAP classes and interfaces. ABAP classes and interfaces are object types defined in Repository Browser.

The following actions by using the Class Builder tool:

- Displaying an overview of global object types and their relationships in Class Browser
- Maintaining the already existing global classes or interfaces easily
- Creating new global classes and interfaces
- Implementing inheritance between two or more global classes
- Creating compound interfaces
- Creating and specifying the attributes, methods, and events of global classes and interfaces

4. FUNCTION BUILDER

- Function Builder plays an important role in defining and maintaining the ABAP functional modules. Function modules are procedures or routines defined in an ABAP program.
- They are stored in function groups, which act as containers for function modules that are logically related to each other.
- The Function Builder tool is used to create a function group and function module.

5. SCREEN PAINTER

- Screen Painter is used to design and manage screens and their elements. It facilitates users to create GUI screens for transactions.

Layout Editor	Manages a screen's layout. The screen layout describes both the screen elements and their layout.
Element List	Manages the ABAP Dictionary or program fields for a screen, and assigns a program field to the OK_CODE (a predefined field name) field in Screen Painter. Screen elements include I/O fields, text fields, check boxes, radio
Attributes	Manage screen's attributes. Screen attributes determine the type of a screen and the program to which the screen belongs.
Flow Logic	Manage screen's flow logic. The flow logic controls the flow of execution of a program.

6. MENU PAINTER

- Menu Painter is used to design user interfaces for ABAP programs.
- The main components of Menu Painter are GUI status, menu bars, menu lists, F-key settings, functions, and titles.

- In other words, using Menu Painter, you can create custom menus in SAP.

7. OBJECT NAVIGATOR

- Object Navigator is another important tool of ABAP Workbench.
- It helps create objects in an SAP system and navigate from one object to another.
- It acts as the central area of ABAP Workbench, where you can access any object of an SAP system.
- Object Navigator perform the following tasks:
    - Select a browser and navigate in the object list
    - Use the tools for development objects
    - Navigate from one window to another window
    - Perform syntax checks in the integrated window
    - Open an object in a new session using an Additional dialog box

8. MESSAGE MAINTENANCE

- Messages help an SAP system communicate with the users.
- For example, error messages display if the users make invalid entries on a screen, making the users aware that a wrong entry has been made. These messages are displayed by using a message class.
- Each message class has an ID and usually contains a set of messages. A message contains a single text line and may contain placeholders for variables.

9. ABAP TEXT ELEMENTS

- The text elements of program texts in different languages can be maintained easily with the help of a tool known as Text Element.
- Creation, maintenance, and translation of the text elements can be done with the help of the Text.
- The following are the various types of text elements:
    - **Text symbols—** Used by the Write statement.
    - **List and column headers**— Appear in an ABAP list.
    - **Selection texts**— Appear on selection screens.

10. MAINTAIN TRANSACTION

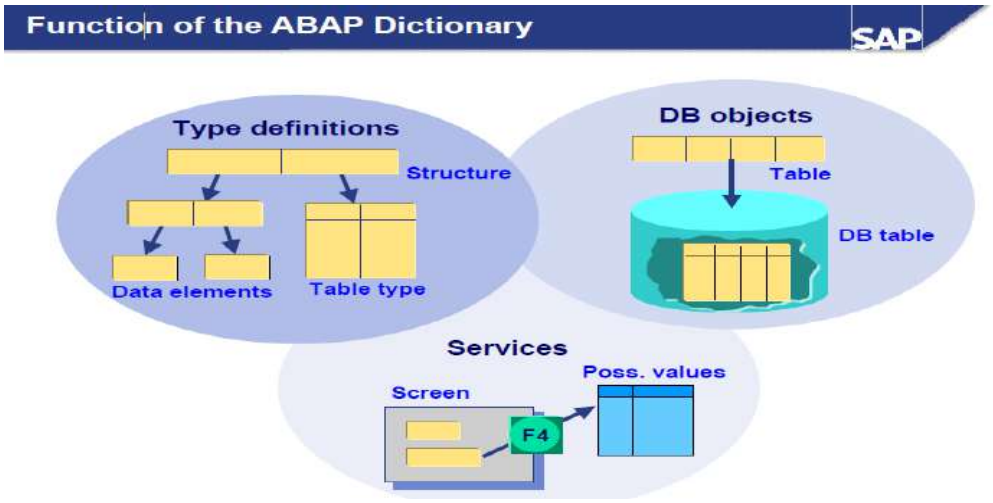
- Maintain Transaction is an ABAP Workbench tool used to create and manage user-defined transactions.

- Transaction screen contains a transaction code name, Ztransaction, in the Transaction Code field.

5. Explain in detail about ABAP/4 data dictionary and repository information.

ABAP/4 Data Dictionary

- The ABAP Dictionary is the central repository for all data definitions in the R/3 system.
- It permits common, non-redundant description of the data.
- It is also used to describe the logical structure of the objects in the application development environment, and shows how they are mapped to structure in the underlying physical database.
- FUNCTIONS OF ABAP DICTIONARY
- In the ABAP Dictionary you can create user-defined types (data elements, structures and table types) for use in ABAP programs or in interfaces of function modules.
- Database objects such as tables and database views can also be defined in the ABAP
- Dictionary and created with this definition in the database.
- The ABAP Dictionary also provides a number of services that support program development.



- Example, setting and releasing locks, defining an input help (F4 help) and attaching a field help (F1 help) to a screen field are supported.

DATABASE OBJECT IN ABAP DICTIONARY

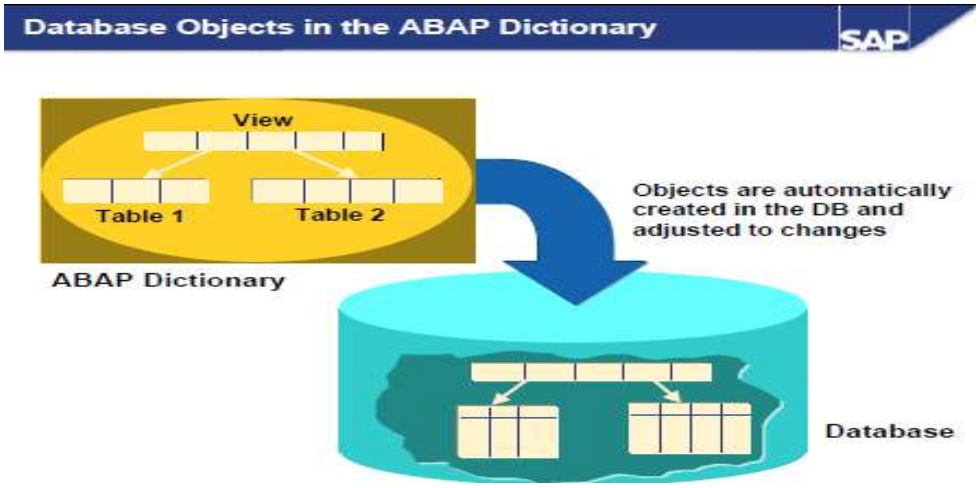
- Tables and database views can be defined in the ABAP Dictionary.
- These objects are created in the underlying database with this definition.
- Changes in the definition of a table or database view are also automatically made in the database.
- Indexes can be defined in the ABAP Dictionary to speed up access to data in a table.

There are three different type categories in the ABAP Dictionary:

- **Data elements:** Describe an elementary type by defining the data type, length
- **Structures:** Consist of components that can have any type.
- **Table types:** Describe the structure of an internal table.
- Any complex user-defined type can be built from these basic types

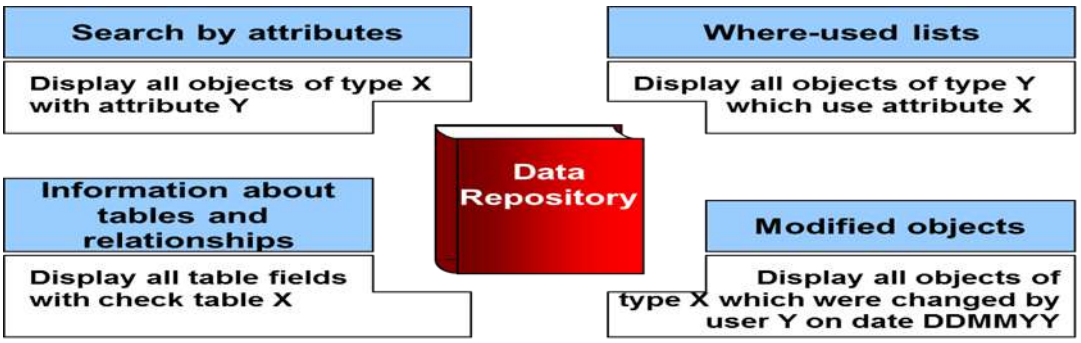
SERVICES OF ABAP DICTIONARY

- The ABAP Dictionary supports program development with a number of services:
- Input helps (F4 helps) for screen fields can be defined with search helps.
- Screen fields can easily be assigned a field help (F1 help) by creating documentation for the data element.
- An input check that ensures that the values entered are consistent can easily be defined for screen fields using foreign keys.
- The ABAP Dictionary provides support when you set and release locks.



ABAP/4 Repository Information

- Data Repository is a Central catalog that contains the descriptions of an organization's data and provides information about the relationships between the data and its use in programs and screens.
- The Repository, the central store for all ABAP workbench development objects, is also cross-client. It contains all Dictionary objects (tables, data elements, domains), and also all ABAP programs, menus and screens. Repository objects are grouped together to form packages.
- A repository is a place where data are stored and maintained.
- The ABAP Repository Information System is a tool which allows you to perform quick searches for information on all ABAP development objects
- The two basic functions available in the Repository Info System are the Find and Where-used list functions.



- The Find function allows you to search for objects from a specific object class meeting selection criteria.
- The Where-used list function allows you to determine the use of an object in other objects. Both functions produce a list of results which met the specified search criteria.
- The ABAP/4 Repository Information System allows you to obtain information about objects (tables, fields, domains, etc.) in the ABAP/4 Repository.

6. Detail about ABAP syntax and datatypes.

ABAP Syntax

- ABAP programs are made up of individual statements.
- The first word in the statement is called an ABAP keyword.
- Each statement must end with a period.
- Statements can be intended.
- Can have multiple statements in a single line
- Every statement provided in the ABAP programming language has a predined syntax associated with it.
- Apart from this syntax, certain conventions need to be followed while writing the syntax for the statements.

Term	Description
Statement	Specifies that ABAP statements and clauses are given in uppercase. It is not mandatory to use the ABAP keywords in uppercase; for instance, the WRITE keyword also can be Write or write.
Operand	Specifies that the operands used in the statement are in lowercase.
[ ]	Specifies that the element enclosed in the square brackets is optional.
| ( )	Specifies that there are elements on both sides of this symbol, and you can use either element in a program. Specifies that parentheses is a part of the syntax of a statement and must be used while using the statement.
,	Separates one or more variables that you need to specify in a statement.
f1, f2	Represents variables indicated with indices. You can list as many variables in a program as you want.
.....	Represents the rest of the code in a program besides the syntax of ABAP statements.

Data Types

- Data types describe data objects. ABAP provides predefined data types that can be used in an ABAP program.
- Apart from the predefined data types, data types can be customized either locally in a program or globally in the SAP R/3 Repository.
- The following are three kinds of data types:
    1. Elementary types
    2. Complex types

3. Reference types

Describing Elementary Types

- Elementary types are the smallest undivided unit of predefined data types. These elementary types are already defined in the SAP system and are visible in all ABAP programs.
- ABAP programs use these predefined elementary types to define local data types (structures) and data objects (fields) used in a program.
- The following are two types of predefined elementary data types, which can be used in an ABAP program:
    i. Fixed-length
    ii. Variable-length
- These types are not derived from other types. The main difference between the fixed-and variable-length types is that the memory space required by the data objects of variable-length data types can change dynamically at runtime.

The Fixed Length Data Type

- As the name suggests, the length of the fixed length data types are fixed at runtime.
- The fixed-length data types are:
    1. Character types
    2. Hexadecimal types
    3. Numeric types

Data Type	Initial Field Length (in bytes)	Valid Field Length (in bytes)	Initial Value	Description
Numeric Types				
I	4	4	0	Specifies an integer (whole number)
F	8	8	0	Represents a floating point number
P	8	1-16	0	Specifies a packed number
Character Type				
C	1	1-65535	'……..'	Denotes a text field (alphanumeric characters)
D	8	8	'00000000	Specifies a date field

				(Format:YYYYMMDD)
N	1	1-65535	'0….0'	Specifies a numeric text field (numeric characters)

Variable-length data type

- A variable-length data type is used for data objects whose length cannot be fixed, as the length varies according to the specified data.
- The variable-length data type is of two types, STRING and XSTRING described in below table.

Variable Length Data Type	Description
STRING	Specifies a sequence of characters with variable lengths. A string can contain any number of alphanumeric characters. The length of the string can be calculated by multiplying the number of characters with the length required for the internal representation of a single character.
XSTRING	Represents a string used for byte strings of a hexadecimal type. A byte string has a variable string and can contain any number of bytes. The length of a byte string is same as the number of bytes.

7. Explicate about constants and variables.

Variables

- Variables are named data objects used to store values within the allotted memory area of a program.
- As the name suggests, users can change the content of variables with the help of ABAP statements.

- Below table shows the statements used to declare a variable statically:

Statement	Description
DATA	Declares the variable whose lifetime is linked to the context of the declaration.
STATICS	Declares the variables that can be used in subroutines, function modules, and static methods.
CLASS-DATA	Declares variables within the classes.
PARAMETERS	Declares the elementary data objects that are linked to input fields on a selection screen.
SELECT-OPTIONS	Declares the internal tables that are linked to input fields on a selection screen
RANGES	Declares internal tables with the same structure as defined in the SELECT- OPTIONS statement, provided the declared internal table is not linked to a selection screen.

- Apart from declaring the variables statically, you can also declare the variables dynamically; that is, whenever you call procedures, such as subroutines.
- For example, the variables declared in the FORM statement (also known as formal parameters) inherit the technical characteristics, such as the data type and length of the parameters defined with the PERFORM statement (also known as actual parameters).
- In addition to this, the values of the actual parameters are assigned to the formal parameters.
- Now, let's discuss the DATA statements in detail.

Data statement

- The DATA statement is used to declare variables in an ABAP program. Variables defined by the DATA statement have a predefined or user-defined data type.

  Syntax

  DATA <f>... [TYPE <type>|LIKE <obj>]... [VALUE <val>].

Description of the clauses of the DATA statement

- **<f>** - Specifies the name of a variable. The name of the variable can be up to 30 characters long.

- **TYPE <type>-** Specifies that the type of <f> variable. Any data type with fully specified technical attributes is known as <type>.
- The possible <type> allotted to a variable can be a non-generic predefined ABAP type (D, F, I, T, STRING, XSTRING) or any existing local data type in a program (the TYPES statement is used to define local data types in a program) or an ABAP Dictionary data type.
- **LIKE <obj>** - Shows that the declared variable name <f> inherits the same technical attributes as an existing data object, <obj>. Predefined data objects that do not need to be declared separately are represented by <obj>.
- **VALUE <val>-** Specifies the initial value of the <f> variable. In case you define an elementary fixed- length variable, the DATA statement automatically populates the value of the variable with the type-specific initial value, as listed in and Other possible values for <val> can be a literal, constant, or an explicit clause, such as IS INITIAL

The following conventions are used while naming a variable in an ABAP program:

- You cannot use special characters such as "+" and "," to name variables.
- The name of the predefined data objects cannot be changed.
- The name of the variable cannot be the same as any ABAP keyword or clause.
- The name of the variables must convey the meaning of the variable without the need for further comments.
- Hyphens are reserved to represent the components of structures.
- Therefore, avoid using hyphens in variable names.
- The underscore character can be used to separate compound words

The following code snippet shows how to define variables by using the DATA statement:

```
DATA      d1(2)   TYPE C.
DATA      d2      LIKE d1.
DATA min_value TYPE I VALUE 10.
```

- In the previous code snippet, d1 is a variable of C type,d2 is a variable of d1 type, and min_value is a variable of I type.

The following code snippet demonstrates another way to declare variables

```
*----------------------------------------------*
*/ Structure Declarations
```

```
*/
TYPES:          BEGIN      OF      number,
                Number_1 TYPE I,
                Number_2 TYPE p DECIMALS 2,
                End of number.
*-----------------------------------------------*
*/Data Declarations
*/
DATA:           n_number TYPE number, Num
          LIKE n_number-Number_1, Date LIKE SY-DATUM,
          Year TYPE i.
```

CONSTANT

- Constants are named data objects created statically by using declarative statements.
- In a program, a constant is declared by assigning a value to it, which is stored in the program's memory area.
- The value assigned to a constant cannot be changed during the execution of the program. When the value of the constant is changed, a syntax or runtime error may occur.
- These named data objects are declared with the help of the CONSTANTS statement.

  The syntax of the CONSTANTS statement is:

  CONSTANTS <f> ... [TYPE <type>|LIKE <obj>]... [VALUE <val>].

- Notice that the syntax of the CONSTANTS statement is similar to the DATA statement.

The following code snippet shows how to define constants by using the CONSTANTS statement:

```
CONSTANTS: abc TYPE P DECIMALS 5 VALUE '1.23456', def TYPE C
VALUE IS INITAIL.
```

The following code snippet shows how to define complex constants:

```
*-----------------------------------------------*
*/ Defining a complex constant
*/
CONSTANTS: BEGIN OF EMPLOYEE,
                Name (20) TYPE c VALUE 'SHIVAM SRIVASTAVA', Company(50)
                TYPE c VALUE 'Software Solutions Incorporation',
```

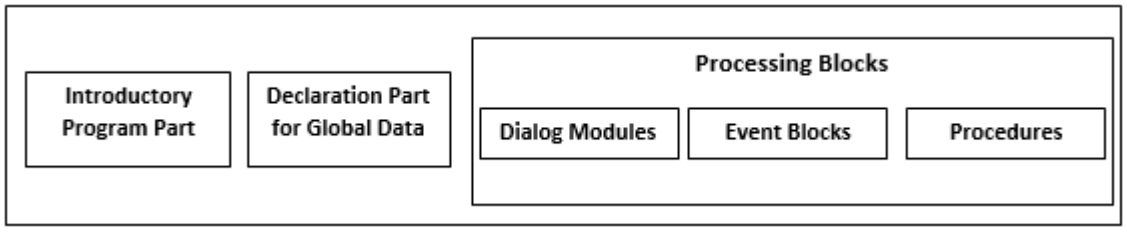
```
                City (10) TYPE c VALUE 'New Delhi', Pincode(8)
                TYPE i VALUE '110002',
            END OF EMPLOYEE.
        *---------------------------------------------*
```

- In the preceding code snippet, EMPLOYEE is a complex constant that is composed of the Name, Company, City, and Pin code fields.

8. Sketch and explain in detail about structure of ABAP/4 program and its types.

Structure of ABAP/4 Program

- ABAP programs are responsible for data processing within the individual dialog steps of an application program. This means that the program cannot be constructed as a single sequential unit, but must be divided into sections that can be assigned to the individual dialog steps.
- To meet this requirement, ABAP programs have a modular structure. Each module is called a processing block.
- A processing block consists of a set of ABAP statements. When you run a program, you effectively call a series of processing blocks. They cannot be nested.
- The following diagram shows the structure of an ABAP program:



Structure of an ABAP program

The structure of an ABAP program includes the following:

1. Introductory program part
2. Global declaration part
3. Processing blocks (consisting of different functions, such as procedures, dialog box, and event blocks)

Introductory Program Part

- Every ABAP program begins with an introductory statement, such as REPORT, PROGRAM, or FUNCTION- POOL.
- Each program has a type, such as executable program, module pool program, or function group program, associated to it.
- The introductory statement of an ABAP program depends on the type of the ABAP program.
- Below table shows the introductory statements for different types of programs.

Introductory Statement	Description of the Program Type
REPORT	Represents an executable program.
PROGRAM	Represents a module pool program and subroutines
FUNCTION-POOL	Represents a function group.
CLASS-POOL	Represents a class pool program.
INTERFACE-POOL	Represents an interface pool program.
TYPE-POOL	Defines the type group.

Declaration of Global data, Class Definitions, and Selection Screens

You always declare global data, classes, and selection screens after the introductory program part of an ABAP program. The declaration of these objects includes:

- **Global data**— Defines global data in an ABAP program. Global data is specified by declaration statements, such as TYPES, TABLES, and DATA. The global data defined in a program is visible in all the internal processing blocks of the program.
- **Selection screens**— Represent special screens that are generated by using ABAP statements instead of Screen Painter. These screens allow you to enter either a single value for one or more fields or a selection criterion.
- **Local class definitions**— Contains the definitions of local classes in an ABAP program. Local class definitions are created with the help of the CLASS DEFINITION statement. The local classes are a part of the ABAP Objects, which are object-oriented extensions of ABAP.

Processing Blocks

- Another part of an ABAP program structure is the processing block.
- A processing block is a set of ABAP statements that represents a module of an ABAP program.
- As we know, ABAP programs are created to process data within the dialog steps of an application
- The second part of an ABAP program contains all of the processing blocks for the program. The following types of processing blocks are allowed:
    1. Dialog modules (no local data area)
    2. Event blocks (no local data area)
    3. Procedures (methods, subroutines and function modules with their own local data area).
- Whereas dialog modules and procedures are enclosed in the ABAP keywords which define them, event blocks are introduced with event keywords and concluded implicitly by the beginning of the next processing block.

Types of ABAP Program

- Each ABAP program has a program type that determines whether a program can be executed.
- An ABAP program can be executed by either entering the program name in the initial screen of ABAP Editor or using a transaction code in the Command field.
- The Type field of ABAP Editor is used to specify the type of a program. The program types that can be selected are:
    1. Executable programs
    2. Module pools
    3. Subroutine pools
    4. Include programs
- Besides the preceding program types, some other types of ABAP programs exist that are not created by ABAP Editor.
- Instead, ABAP Workbench provides specific tools to create and maintain these programs. These program types are:
    1. Function pools
    2. Class pools

3. Interface pools

Defining Executable Programs

- Executable programs are often called report programs because the source code of an executable program starts with the REPORT statement.
- These programs are created by processing the data stored in an SAP database.
- Executable programs cannot only retrieve the data from the database but also modify the data stored in the database.
- Executable programs can contain almost every kind of processing block, such as dialog modules, methods, and event blocks, except function modules and local classes.

Defining Module Pools

- A module pool is a program used to display data in and add functionality to a screen, such as a button, radio button, group box, or menu bar. You can write the screen flow logic using a module pool program; a subroutine is not used for this purpose

Defining Subroutine Pools

- A subroutine pool is a program that contains a collection of subroutines and local or global classes and interfaces that must be called externally in other ABAP programs.

Defining Include Programs

- Include programs are not complete programs because they do not have a memory and cannot be executed, unlike other stand-alone programs

Defining Function Pools

- Function pools, called function groups, are programs that contain function modules. A function pool program is created by using the FUNCTION-POOL statement.
- Each function pool contains global data, such as data objects, subroutines, or screens, which are shared by all the function modules of the function pool.

Defining Class Pools

- A class pool is a special ABAP program used to store global classes and interfaces. All the ABAP programs can access these global classes and interfaces.
- A class pool is created by using the CLASS-POOL statement or by using.

Defining Interface Pools

- Interface pools are programs that store global interfaces. An interface pool acts as a container that can store exactly one global interface. You can use an interface pool to implement the methods, which are defined in an interface of a class.

9. List out and explain the ABAP statements.

There are ten ABAP statements are available. They are

1. DATA
2. PARAMETERS
3. TABLE
4. MOVE
5. MOVE-CORRESPONDING
6. CLEAR
7. WRITE
8. CHECK
9. FORMAT
10. LOOP STRUCTURES

1. DATA

- The DATA statement is used to declare variables in an ABAP program. Variables defined by the DATA statement have a predefined or user-defined data type.

  Syntax

  DATA <f>... [TYPE <type>|LIKE <obj>]... [VALUE <val>].

Description of the clauses of the DATA statement

- **<f>** - Specifies the name of a variable. The name of the variable can be up to 30 characters long.
- **TYPE <type>-** Specifies that the type of <f> variable. Any data type with fully specified technical attributes is known as <type>.
- The possible <type> allotted to a variable can be a non-generic predefined ABAP type (D, F, I, T, STRING, XSTRING) or any existing local data type in a program (the TYPES statement is used to define local data types in a program) or an ABAP Dictionary data type.
- **LIKE <obj>** - Shows that the declared variable name <f> inherits the same technical attributes as an existing data object, <obj>. Predefined data objects that do not need to be declared separately are represented by <obj>.
- **VALUE <val>-** Specifies the initial value of the <f> variable. In case you define an elementary fixed- length variable, the DATA statement automatically populates the value of the variable with the type-specific initial value, as listed in and Other possible values for <val> can be a literal, constant, or an explicit clause, such as IS INITIAL

The following conventions are used while naming a variable in an ABAP program:

- You cannot use special characters such as "+" and "," to name variables.
- The name of the predefined data objects cannot be changed.
- The name of the variable cannot be the same as any ABAP keyword or clause.
- The name of the variables must convey the meaning of the variable without the need for further comments.
- Hyphens are reserved to represent the components of structures.
- Therefore, avoid using hyphens in variable names.
- The underscore character can be used to separate compound words

The following code snippet shows how to define variables by using the DATA statement:

```
DATA      d1(2)   TYPE C.
DATA      d2      LIKE d1.
DATA min_value TYPE I VALUE 10.
```

- In the previous code snippet, d1 is a variable of C type,d2 is a variable of d1 type, and min_value is a variable of I type.

The following code snippet demonstrates another way to declare variables

```
*---------------------------------------------*
```

```
*/ Structure Declarations
*/
TYPES:          BEGIN      OF      number,
                Number_1 TYPE I,
                Number_2 TYPE p DECIMALS 2,
                End of number.
*----------------------------------------------*
*/Data Declarations
*/
DATA:           n_number TYPE number, Num
          LIKE n_number-Number_1, Date LIKE SY-DATUM,
          Year TYPE i.
```

2. PARAMETER

- The PARAMETERS statement is used to enter the values of the variables on both the standard selection screen as well as user-defined selection screens.
- These variables (also known as parameters) are defined with the help of the PARAMETERS statement.
- Each parameter defined with the PARAMETERS statement appears as an input field on the relevant selection screen.
- The PARAMETERS statement is used to enter single values in the input fields.
- The flow of the program can be controlled with the help of parameters.

Syntax

PARAMETERS <p> [(<length>)] [TYPE <type>|LIKE <obj>] [DECIMALS <d>]

- In the preceding syntax, the use of the TYPE or LIKE clause is optional, but the data must be of the C type with length 1.

Describes the clauses used in the PARAMETERS statement:

- **<p>** - Represents the name of a parameter. The maximum length for naming parameters is 8 instead of 30, as in the DATA statement.
- **<length>** - Specifies the length of a variable.
- **TYPE <length>** - Specifies a data type with fully specified technical attributes. The possible <type> allotted to a variable can be a predefined ABAP type (D, I, T, STRING, XSTRING) or an ABAP Dictionary data type. The TYPE keyword shows that the declared parameter name <p> inherits the same technical attributes as that of an existing data type <type>.-

- **LIKE <obj>** - Specifies a predefined data object that is already present and need not be declared separately, such as system variables. Fields used to provide information about the current state of the SAP system are known as system variables. The LIKE keyword specifies that the declared parameter name <p> inherits the same technical attributes as that of the existing data object <obj>.
- **DECIMALS <d>** - Specifies the number of decimal places for numeric values.

Sample program using Using the PARAMETERS statement

```
REPORT ZPARAMETERDISPLAY. *----------------------------------------------*
*/Creating Parameters
*/
PARAMETERS: NAME (10) TYPE C, CLASS TYPE I,
SCORE TYPE P DECIMALS 2,
CONNECT TYPE MARA-MATNR.
*----------------------------------------------*
```

Four parameters are created and displayed on the standard selection screen. The parameters are:

- NAME—Represents a parameter of 10 characters
- CLASS—Specifies a parameter of integer type with the default size in bytes
- SCORE—Represents a packed type parameter, with values up to two decimal places
- CONNECT—Refers to the MARA-MATNR type of ABAP Dictionary

3. TABLE

- The TABLES statement is used to create a structure having the same name as a database table, view, or structure defined in ABAP Dictionary.
- This statement actually defines a table work area, which is a kind of interface work area, used to create in the shared area of a program.

  Syntax

  TABLES <dbtab>.
- In the preceding syntax, <dbtab> represents a structure with the same data type and name of a database table, a view, or a structure from ABAP Dictionary.

4. MOVE

- The MOVE statement is used to assign the value of a data object to another data object; that is, to transfer the content of one field to another.

  Syntax

  MOVE <f1> TO <f2>.

- In this syntax, <f1> is the data object whose data has to be transferred to another data object, <f2>. The equivalent statement for the MOVE statement is <f2> = <f1>. The MOVE statement assigns the value of one data object to another, but the value of the original data object remains unchanged.
- You can use the MOVE statement to assign values to multiple data objects, such as

  <f4> = <f3> = <f2> = <f1>.
- The following syntax shows how to use the MOVE statement for multiple assignments:

  MOVE <f1> TO <f2>.

  MOVE <f2> TO <f3>.

  MOVE <f3> TO <f4>.

Sample program using MOVE statement

```
REPORT ZMOVE_DEMO.
*---------------------------------------------*
*/ Data Declarations
*/
DATA: Number_1(10) TYPE c,
Number_2 TYPE p DECIMALS 2, Number_3 TYPE i.
*---------------------------------------------* Number_1 = 100.
*---------------------------------------------**
/MOVE statement */
MOVE '5.75' TO Number_2.
*---------------------------------------------* Number_3 = Number_1.
WRITE: 'Number_1 =',Number_1.
WRITE: / 'Number_2 =',Number_2. WRITE: / 'Number_3 =',Number_3.
```

5. MOVE CORRESPONDENCE

- The MOVE-CORRESPONDING statement is used to assign values between the components of two or more structures.

Syntax

MOVE-CORRESPONDING <struct1> TO <struct2>.

6. CLEAR

- The CLEAR statement is used to reset the value of a data object. Resetting the values of the data objects depends on the data type of the data objects.
- For example, data objects of type I are set to zero and data objects of type C are set to null.

Syntax

CLEAR <f>.

Sample program using CLEAR statement

```
REPORT ZCLEAR.
*---------------------------------------*
*/ Data Declarations
*/
DATA number TYPE I VALUE '80'.
*---------------------------------------* WRITE number.
*---------------------------------------**
/Using CLEAR statement
CLEAR number. WRITE / number.
*---------------------------------------*
```

7. WRITE

WRITE TO

- The WRITE TO statement is an assignment statement that converts the content of the source field into a field of type C.

Syntax

WRITE <f1> TO <f2> [<option>].

- In the preceding syntax, the WRITE TO statement converts the content of the data object <f1> to type C and places the string into the <f2> variable. Listing 6.7 shows the use of the WRITE TO statement

Sample program using WRITE TO statement

```
REPORT ZWRITE_DEMO.
*-------------------------*
```

```
*/ Data Declarations
*/ DATA: Name(10) TYPE C VALUE 'SHIVAM', TEXT(10).
*----------------------------*
*/Using WRITE TO statement to show source and target names
*/
WRITE :'Source name is :', Name.WRITE: Name TO TEXT. WRITE: /'Target
name is :', TEXT.
*----------------------------*
```

WRITE

- The WRITE statement is a formatting statement used to display data on a screen. There are different formatting options for the WRITE statement.

Syntax

WRITE <format> <f> <options>.

8. FORMAT

- Different types of syntaxes are available for specific actions performed with the FORMAT statement.
- **FORMAT <option1> [ON|OFF] <option2> [ON|OFF]..... -**Specifies that the formatting options are set statically in a program. The option mentioned in the <option> expression is applicable to all output untill the formatting option is turned off by using the OFF clause, as indicated in the statement. Note that the ON or OFF clause is optional.
- **FORMAT <option1> = <var1> <option2> =<var2>....-**Specifies that the formatting options are set dynamically at runtime. The variables var1, var2, ... are interpreted as numbers and should be declared with the data type as I. If the content in the var1 and var2 variables is zero, the variable has the same effect as the OFF clause. If the numbers in the var1 and var2 variables are not equal to zero, either the color is shown according to the numbers stored in the variables or the variables have the same effect as that of the ON clause. Note that the formatting options are all set to their default values whenever they are used for the new event.
- **FORMAT RESET -**Sets all the formatting options to off.

Using the FORMAT Statement Clauses

- The FORMAT statement is also used to display the output in a colored format.

  Syntax

  FORMAT COLOR <num> [ON] INTENSIFIED [ON|OFF] INVERSE [ON|OFF].

The following syntax is used to set colors at runtime:

FORMAT COLOR = <const> INTENSIFIED = <int> INVERSE = <inv>.

- In both syntaxes, the COLOR clause is used to set the background color of a line

9. CHECK

- Conditionally leaves a loop or processing block.

  Syntax

  CHECK <logexp>.

- If the logical expression <logexp> is true, the system continues with the next statement. If it is false, processing within the loop is interrupted at the current loop pass, and the next loop pass is performed. Otherwise the system leaves the current processing block.
- In conjunction with selection tables, and inside GET events, you can use an extra variant of the CHECK statement.

10. LOOPING

- A statement block is executed repeatedly in a program by using loops in the program. The loops are categorized into two types:
  i. Unconditional loops by using the DO statement.
  ii. Conditional loops by using the WHILE statement

Unconditional Loops Using the DO Statement

- Unconditional loops repeatedly execute several statements without specifying any condition.
- The DO statement implements unconditional loops by executing a set of statements (statement block) several times unconditionally.

Syntax

DO [n TIMES] [VARYING <f> FROM <fi> NEXT <f2>].

<statement block>

ENDDO.

Sample program using DO statement

REPORT demo_do.

***/Using DO Statement**

***/ DO.**

WRITE / SY-INDEX. IF SY-INDEX = 3. EXIT.

ENDIF.

ENDDO.

Conditional Loops Using the WHILE Statement

- Conditional loops execute several statements only after certain conditions are fulfilled. This can be accomplished with the WHILE statement.
- The WHILE statement allows you to execute a block of statements so long as the condition mentioned in the WHILE statement evaluates to true.

  Syntax

  WHILE <condition> [VARY <f> FROM <f1> NEXT <f2>]

  [statement_block]

  ENDWHILE.
- You can nest the WHILE loop any number of times and combine the nested WHILE loop with other loop forms.

TERMINATING LOOP

- A loop can be terminated prematurely with the help of two types of termination statements provided by
- ABAP. The two types of termination statements are:
  1. Statements that apply to a loop, such as CONTINUE, CHECK, and EXIT.
  2. Statements that apply to an entire processing block, such as STOP and REJECT.

- The CONTINUE statement can be used only in a loop statement. On the other hand, the CHECK and EXIT statements are context-sensitive. When used in the loop, the CHECK and EXIT statements are responsible for the execution of the loop itself. However, outside the loop, the statements terminate the entire processing block, such as a subroutine, dialog module, or event block, in which they occur.
- CONTINUE, CHECK, and EXIT can be used in looping statements, such as DO, WHILE, LOOP, and SELECT.